LAB-06 MASTER REVISION HANDBOOK

Chapter 1: Goal of Lab-06
Lab-06 is the transition point between transaction-level verification and signal-level verification. In previous labs we created packets, sequences, tests, factory overrides and sequence libraries. However, packets only existed as SystemVerilog class objects. The DUT cannot understand class objects. The DUT only understands HDL signals. Therefore Lab-06 introduces interfaces, virtual interfaces, driver BFMs, monitor BFMs and DUT connectivity.

Main Objective
Convert UVM transactions into actual signal activity, transmit them through the router interface, observe them at signal level and reconstruct them back into transaction objects.

Chapter 2: Big Picture Understanding
Before Lab-06:
Sequence -> Driver -> Print Packet

After Lab-06:
Sequence -> Sequencer -> Driver -> Interface -> DUT -> Monitor -> Packet Reconstruction

This is the architecture used in real industrial UVM environments.

Chapter 3: Core Concepts Introduced
1. Interface: Bundle of related DUT signals.
2. Virtual Interface: Mechanism allowing classes to access HDL signals.
3. Driver BFM: Converts Packet -> Signals.
4. Monitor BFM: Converts Signals -> Packet.
5. Transaction Recording: begin_tr()/end_tr().
6. DUT Integration: Connecting verification environment to RTL.

Chapter 4: Interface Creation
Interface Created:
```
interface yapp_if(input logic clock,input logic reset);
logic [7:0] in_data;
logic in_data_vld;
logic in_suspend;
endinterface
```

Why?
To group all YAPP protocol signals into a single reusable object.

Learning:
Interface = Container of Signals.

Chapter 5: Virtual Interface Flow
Top creates interface instance.
Config DB passes interface handle.
Driver and Monitor retrieve handle using get().

Why Virtual Interface?
SystemVerilog classes cannot directly access HDL signals. Virtual interfaces bridge this gap.

Memory Trick:
Interface = Real Road.
Virtual Interface = GPS location of the road for UVM classes.

Chapter 6: Driver BFM Development

Old Driver:
get_next_item()
print packet
item_done()

New Driver:
get_next_item()
send_to_dut()
item_done()

Driver Responsibilities:
1. Wait for reset release.
2. Get packet from sequencer.
3. Drive header.
4. Drive payload.
5. Drive parity.
6. Handle packet delay.
7. Handle suspend conditions.
8. Record transaction.

Learning:
Driver = Transaction to Signals conversion engine.

Chapter 7: Packet Transmission Flow
Header = {Length, Address}
Payload = Data bytes
Parity = Error checking byte

Transmission Order:
Header -> Payload -> Parity

Learning:
Monitor must reconstruct data in the same order.

Chapter 8: Monitor BFM Development
Monitor waits for packet start using in_data_vld.
Monitor reconstructs header.
Monitor allocates payload memory.
Monitor captures payload bytes.
Monitor captures parity.
Monitor checks parity correctness.

Learning:
Driver and Monitor are mirror images.

Driver: Packet -> Signals
Monitor: Signals -> Packet

Chapter 9: Transaction Recording
Driver:
begin_tr()
end_tr()

Monitor:
begin_tr()
end_tr()

Purpose:
Visual debugging and transaction tracking inside waveform viewers.

Chapter 10: Router Testbench Layer
Router TB was introduced between Test and Environment.

Reason:
To isolate DUT connectivity from reusable verification components.

Architecture:
Test -> Router_TB -> Env -> Agent

Chapter 11: DUT Integration
Router DUT instantiated in top.
Input interface connected to DUT input ports.
Output channels connected to DUT output ports.

Learning:
Verification environment is now exercising real RTL.

Chapter 12: Complete Debugging Journey
Issue 1: Simulation ended at time 0.
Root Cause: No objections raised.
Fix: raise_objection()/drop_objection().
Learning: Drain time does not keep simulation alive.

Issue 2: Reset dropped message never appeared.
Root Cause: Simulation ended before reset deassertion.
Fix: Hold run phase with objections.

Issue 3: Driver sent 0 packets.
Root Cause: Driver never got opportunity to run after reset.
Fix: Objection mechanism corrected.

Issue 4: Driver got item but packet never transmitted.
Root Cause: Driver stuck at suspend wait condition.

Issue 5: in_suspend signal bug.
Root Cause: top variable and interface signal were different objects.
Fix: Drive interface signal directly using in0.in_suspend.
Learning: Signal existence does not imply connectivity.

Issue 6: Monitor collected only one packet.
Root Cause: collect_packet() called once.
Fix: forever loop around collect_packet().

Issue 7: Monitor reused packet object.
Root Cause: Same transaction handle reused repeatedly.
Fix: Create packet inside collect_packet().

Chapter 13: Real Project Importance
Every industrial UVM environment uses:
Virtual Interfaces
Driver BFMs
Monitor BFMs

Transaction Recording
Protocol Monitoring
RTL Connectivity

Without Lab-06 concepts, verification environments cannot communicate with RTL.

Chapter 14: Interview Questions
What is a Virtual Interface?
Why use an Interface?
Difference between Driver and Monitor?
What is a BFM?
Why wait for reset release?
Why create packets inside monitor collection logic?
Why are objections required?
What is transaction recording?

Chapter 15: Final Revision Summary
Interface = Bundle of Signals.
Virtual Interface = Access mechanism for classes.
Driver = Packet -> Signals.
Monitor = Signals -> Packet.
Router TB = DUT connectivity layer.
Transaction Recording = Debug support.
Objections control simulation lifetime.
Signal connectivity bugs are common and must be debugged carefully.

Memory Map
Lab-01 = Build Packet
Lab-02 = Build Architecture
Lab-03 = Control Environment
Lab-04 = Factory Override
Lab-05 = Generate Traffic
Lab-06 = Put Traffic On Signals and Connect to DUT

One-Line Master Revision
Lab-06 teaches how UVM transactions are converted into real protocol signals, driven into a DUT, monitored back into transactions, debugged through BFMs and transaction recording, and verified in a realistic signal-level environment.